This single line defines routes for `index`, `store`, `show`, `update`, and `destroy` actions. You can also group these routes with middleware, such as authentication, for better control and security.

14.4 Building the Product API

Begin by creating a `Product` model along with its migration file:

```
php artisan make:model Product -m
```

In the generated migration file, add the necessary fields such as `name`, `description`, `price`, and `stock`. After defining the schema, run the migration:

```
php artisan migrate
```

Next, create a controller to handle API logic:

```
php artisan make:controller API/ProductController --api
```

This generates a resource controller with all necessary methods. You can now implement logic in each method to perform CRUD operations using the `Product` model.

14.5 Using JSON Resources for API Responses

Laravel offers a powerful way to format API responses using Resource classes. To create a resource for the `Product` model, use the following command:

```
php artisan make:resource ProductResource
```

In the `ProductResource` class, define how the model should be represented in the API response. This allows for a consistent and clean structure of your JSON output:

```
public function toArray($request)
{
    return [
        'id' => $this->id,
        'name' => $this->name,
        'price' => $this->price,
        'stock' => $this->stock,
        'created_at' => $this->created_at->toDateString(),
    ];
}
```

Using resources ensures that your API responses remain consistent across endpoints and are easy to maintain.

14.6 Input Validation and Error Handling